Deep Agents Deployment

Context Management for Deep Agents

The LangChain Team
January 28, 2026

6 min

By Chester Curme and Mason Daugherty

As the addressable task length of AI agents continues to grow, effective context management becomes critical to prevent context rot and to manage LLMs' finite memory constraints.

The Deep Agents SDK is LangChain's open source, batteries-included agent harness. It provides an easy path to build agents with the ability to plan, spawn subagents, and work with a filesystem to execute complex, long-running tasks. Because these sorts of tasks can generally exceed models' context windows, the SDK implements various features that facilitate context compression.

Context compression refers to techniques that reduce the volume of information in an agent's working memory while preserving the details relevant to completing the task. This might involve summarizing previous interactions, filtering out stale information, or strategically deciding what to retain and what to discard.

Deep Agents implements a filesystem abstraction that allows agents to perform operations such as listing, reading, and writing files, as well as search, pattern matching, and file execution. Agents use the filesystem to search and retrieve offloaded content as needed.

Deep Agents implements three main compression techniques, triggered at different frequencies:

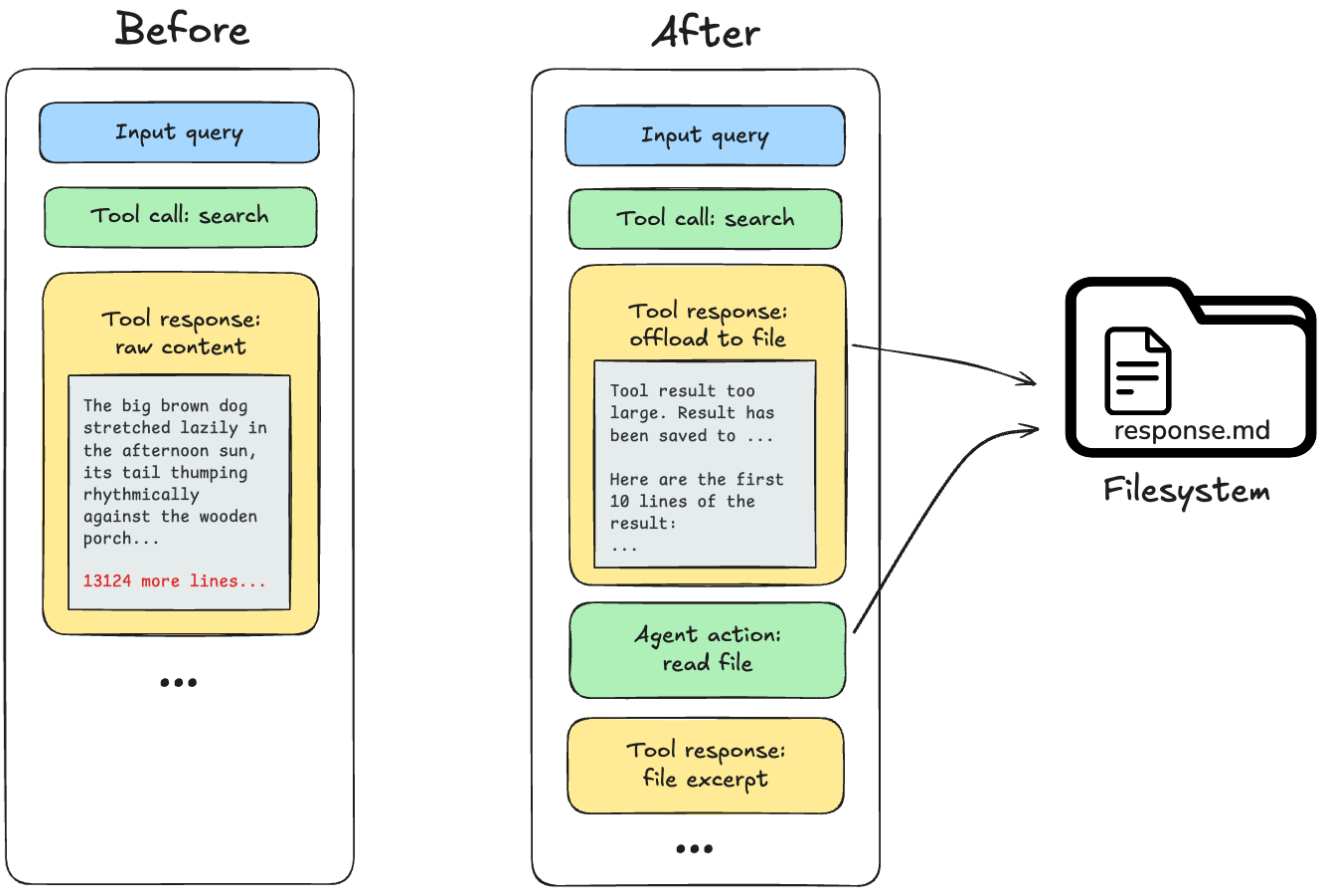
1. **Offloading large tool results:** We offload large tool responses to the filesystem whenever they occur.
2. **Offloading large tool inputs:** When the context size crosses a threshold, we offload old write/edit arguments from tool calls to the filesystem.

3. **Summarization**: When the context size crosses the threshold, and there is no more context eligible for offloading, we perform a summarization step to compress the message history.

To manage context limits, the Deep Agents SDK triggers these compression steps at threshold fractions of the model's context window size. (Under the hood, we use LangChain's model profiles to access the token threshold for a given model.)

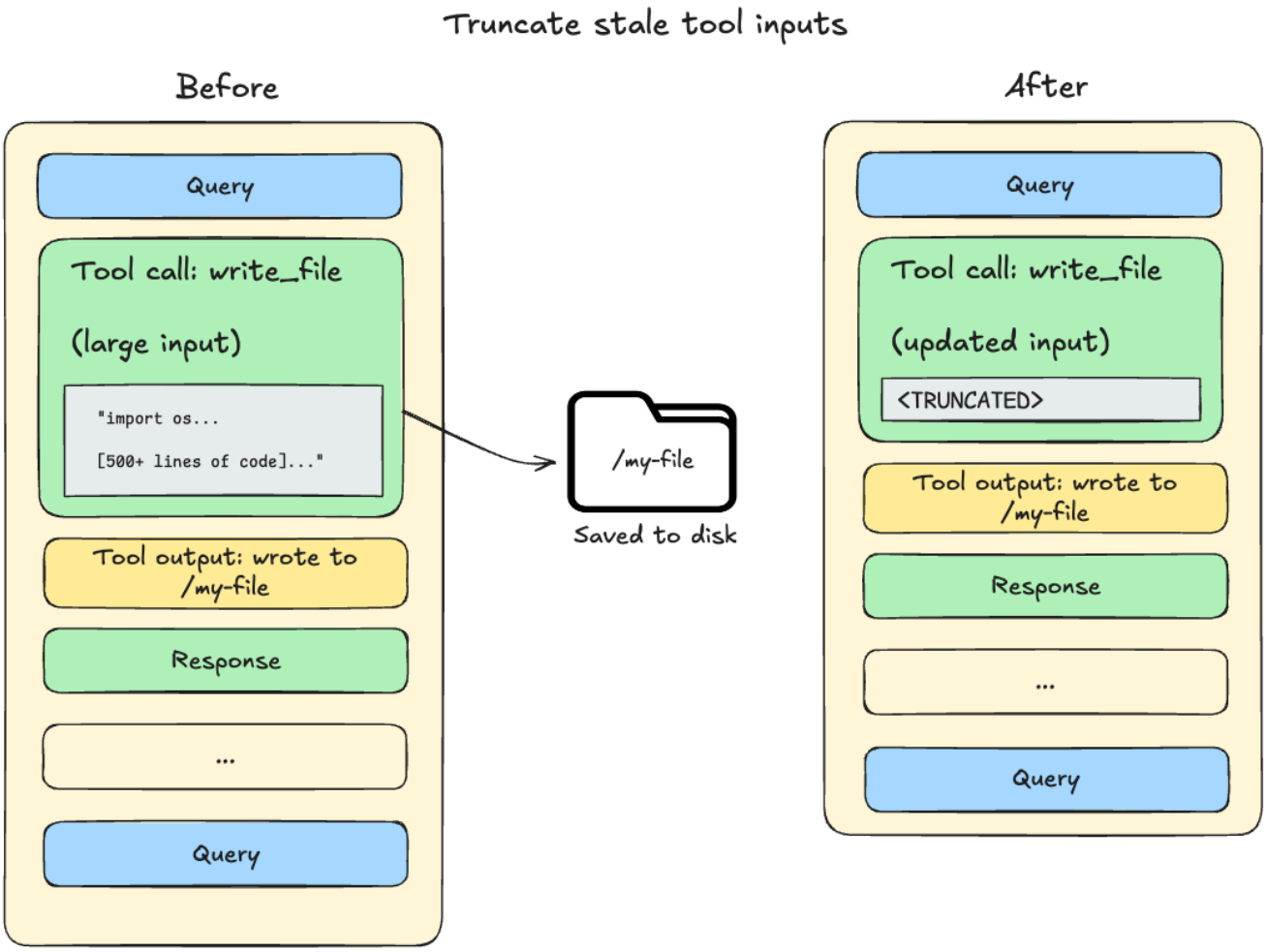
Offloading large tool results

Responses from tool invocations (e.g., the result of reading a large file or an API call) can exceed a model's context window. When Deep Agents detects a tool response exceeding 20,000 tokens, it offloads the response to the filesystem and substitutes it with a file path reference and a preview of the first 10 lines. Agents can then re-read or search the content as needed.



Offloading large tool inputs

File write and edit operations leave behind tool calls containing the complete file content in the agent's conversation history. Since this content is already persisted to the filesystem, it's often redundant. As the session context crosses 85% of the model's available window, Deep Agents will truncate older tool calls, replacing them with a pointer to the file on disk and reducing the size of the active context.

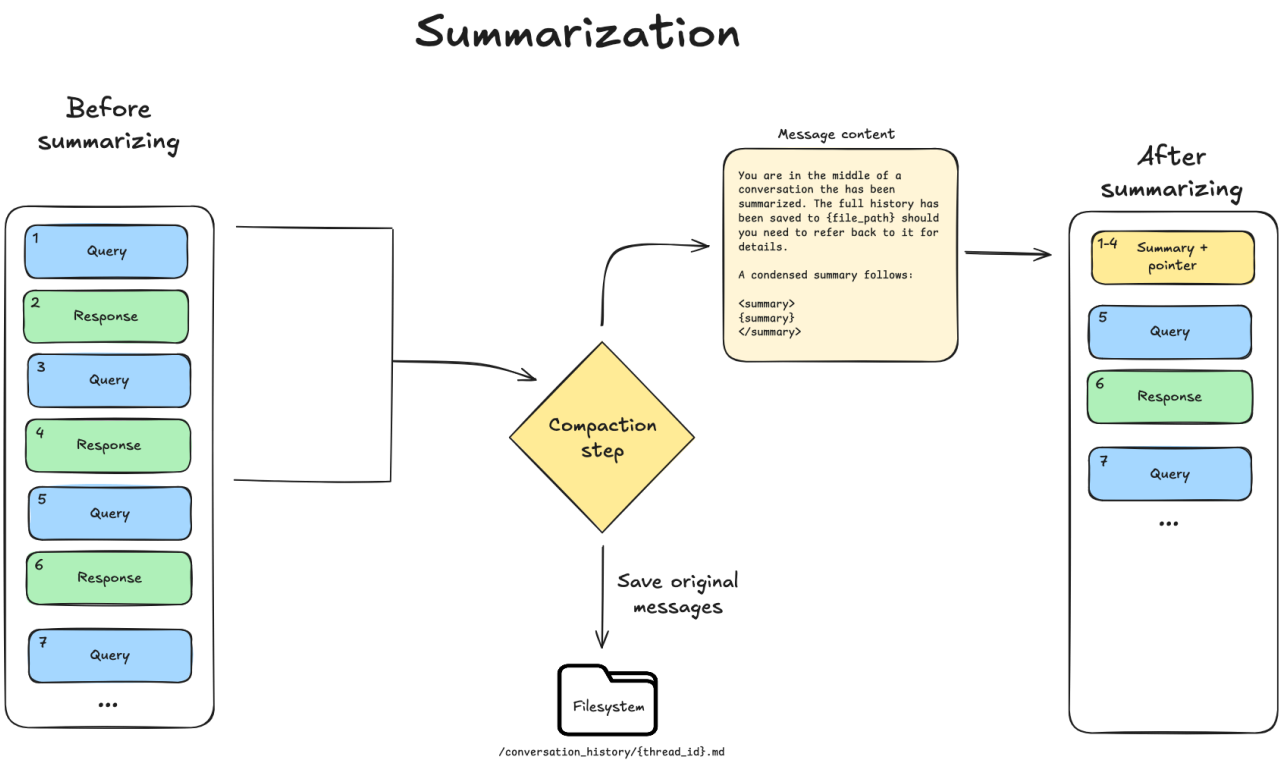


Summarization

When offloading no longer yields sufficient space, Deep Agents falls back to summarization. This process has two components:

1. In-context summary: An LLM generates a structured summary of the conversation—including session intent, artifacts created, and next steps—which replaces the full conversation history in the agent's working memory. (See the Deep Agents summarization prompt.)
2. Filesystem preservation: The complete, original conversation messages are written to the filesystem as a canonical record.

This dual approach ensures the agent maintains awareness of its goals and progress (via the summary) while preserving the ability to recover specific details when needed (via filesystem search). See an example in this trace, where the model uses the `read_file` tool to fetch previously offloaded messages.



What this looks like in practice

While the techniques above provide the machinery for context management, how do we know they're actually working? Runs on real-world tasks, as captured in benchmarks such as `terminal-bench` , may trigger context compression sporadically, making it difficult to isolate their impact.

We've found it useful to increase the signal of individual features of the harness by engaging them more aggressively on benchmark datasets. For example, while triggering summarization at 10 - 20% of the available context window may lead to suboptimal overall performance, it produces significantly more summarization events. This allows for different configurations (e.g., variations of your implementation) to be compared. For example, by forcing the agent to summarize frequently, we could identify how simple changes to the `deepagents` summarization prompt, in which we added dedicated fields for the session intent and next steps, help improve performance.

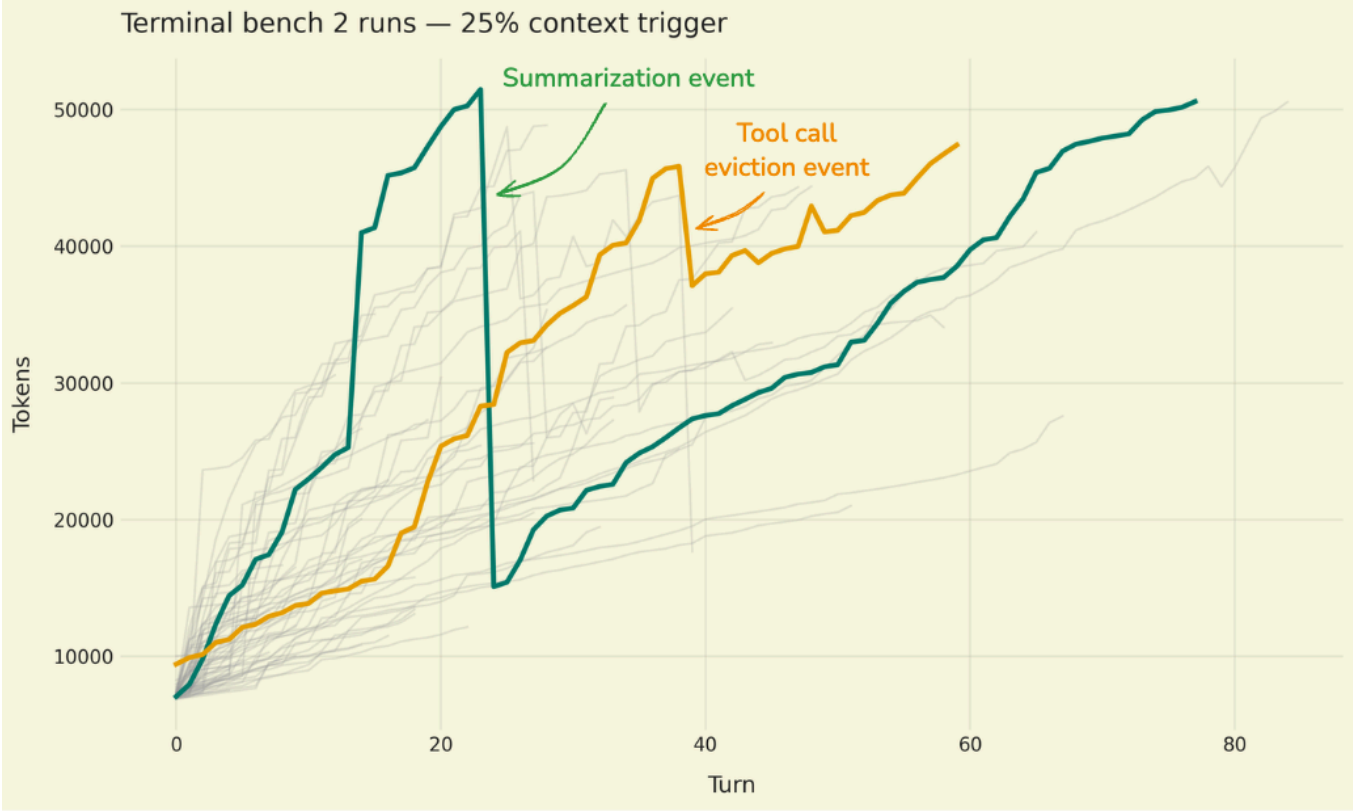


Figure: Token usage over time in sample runs of Claude Sonnet 4.5 on terminal-bench-2 (gray lines show all runs; colored lines highlight two specific examples). The green line shows a dramatic token drop around turn 20 when a summarization event compresses the conversation history. The orange line shows a smaller reduction around turn 40 when a large file write tool call is evicted from context. By triggering compression at 25% of the context window (rather than the Deep Agents default of 85%), we generate more events to study.

Targeted evals

The Deep Agents SDK maintains a set of targeted evaluations designed to isolate and validate individual context-management mechanisms. These are deliberately small tests that make specific failure modes obvious and debuggable.

The goal of these evals is not to measure broad task-solving ability, but to ensure that the agent's harness does not get in the way of certain tasks. For example:

- Did summarization preserve the agent's objective? Some evals deliberately trigger summarization mid-task and then check whether the agent continues. This ensures that summarization preserves not only agent state but also its trajectory.
- Can the agent recover information that was summarized away? Here we embed a "needle-in-the-haystack" fact early in the conversation, force a summarization event, and then require the agent to recall that fact later to complete the task. The fact is not present in the active context after summarization and must be recovered via filesystem search.

These targeted evals act as integration tests for context management: they don't replace full benchmark runs, but they significantly reduce iteration time and make failures attributable to specific compression mechanisms rather than overall agent behavior.

Guidance

When evaluating your own context compression strategies, we'd emphasize:

- **Start with real-world benchmarks, then stress-test individual features.** Run your harness on representative tasks first to establish baseline performance. Then, artificially trigger compression more aggressively (e.g., at 10-20% of context instead of 85%) to generate more compression events per run. This amplifies the signal from individual features, making it easier to compare different approaches (e.g. variations in your summarization prompt).
- **Test recoverability.** Context compression is only useful if critical information remains accessible. Include targeted tests that verify agents can both continue toward their original goal after compression and recover specific details when needed (e.g., needle-in-the-haystack scenarios where a key fact is summarized away but must be retrieved later).
- **Monitor for goal drift.** The most insidious failure mode is an agent that loses track of the user's intent after summarization. This may manifest as the agent completing in the turn after summarization to ask for clarification, or to mistakenly declare the task complete. More subtle deviations from the intended task may be harder to attribute to summarization; forcing frequent summarization on sample datasets may help surface these failures.

All features of the Deep Agents harness are open source. Try out the latest version and let us know what compression strategies work best for your use cases!